Module 13)>>3. Core Python Concepts

- **Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.**

1. Integers (int)

- Whole numbers (positive or negative) without a decimal point.
- Example: 5, -3, 0, 1000

2. **Floats (**float**)**

- Numbers with decimal points (real numbers).
- Example: 3.14, -0.001, 2.0

3. **Strings (**str**)**

- Ordered sequences of characters enclosed in quotes (' ' or " ").
- Example: "hello", 'Python', "123" (as text)
- Strings are **immutable** (cannot be changed after creation).

4. **Lists (**list**)**

- Ordered, mutable (changeable) collections of items (can be mixed types).
- Enclosed in square brackets [ ].
- Example: [1, 2, 3], ["apple", "banana", 3.14]
- Supports indexing (list[0]) and slicing (list[1:3]).

5. **Tuples (**tuple**)**

- Ordered, **immutable** (unchangeable) collections of items.
- Enclosed in parentheses ( ).
- Example: (1, 2, 3), ("apple", "banana", 3.14)
- Faster than lists for fixed data

6. **Dictionaries (**dict**)**

- Unordered collections of key-value pairs (keys must be unique and immutable).
- Enclosed in curly braces { } with key: value pairs.
- Example: {"name": "Alice", "age": 25, "city": "New York"}
- Accessed via keys (dict["name"]).

7. **Sets (**set**)**

- Unordered collections of unique elements (no duplicates).
- Enclosed in curly braces { } (but without key:value pairs).
- Example: {1, 2, 3}, {"apple", "banana"}
- Useful for membership tests and eliminating duplicates.

• Python variables and memory allocation

1. Variable Assignment

- A variable is created when you assign a value to it.
- Python variables do not need explicit declaration.
- The same variable can be reassigned to different data types.

```
x = 10          # x is an integer
x = "hello"     # Now x is a string
x = [1, 2, 3]   # Now x is a list
```

2. How Python Allocates Memory

- When you assign a variable, Python:
    1. Creates an object in memory (10, "hello").
    2. Binds the variable name to that object.

3. Uses reference counting to track how many variables point to the same object.

```
a = 5      # Memory allocated for integer '5', 'a' points to it
b = a      # 'b' now also points to the same '5'
b = 10     # 'b' now points to a new object '10', 'a' still points to '5'
```

3. Memory Management: Garbage Collection

- Python uses automatic garbage collection to free memory when an object is no longer referenced.
- An object is deleted when its reference count drops to zero.
- The del keyword can manually remove a reference.

4. Variable Storage: Stack vs. Heap

- Stack: Stores variable names and references (fast access).
- Heap: Stores actual objects (lists, dictionaries, etc.).
- Variables are pointers to objects in the heap.

5. Mutable vs. Immutable Objects

- Immutable objects (int, float, str, tuple) cannot be modified after creation.

```
a = 5
b = a   # Both point to the same '5'
b += 1  # 'b' now points to a new '6', 'a' remains '5'
```

6. Checking Memory Address (id()**)**

- The id() function returns the memory address of an object.
- If two variables have the same id(), they point to the same object.

7. Memory Optimization (Interning)

- Python interns small integers (typically -5 to 256) and some strings to save memory.

- **Python operators: arithmetic, comparison, logical, bitwise.**

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2 (integer division)
%	Modulus	5 % 2	1 (remainder)
**	Exponentiation	2 ** 3	8 (2^3)

Example:

```
print(10 + 3)   # 13
print(10 / 3)   # 3.333... (float division)
print(10 // 3)  # 3 (integer division)
print(2 ** 4)   # 16 (2^4)
```

2. Comparison (Relational) Operators

Used to compare two values (return True or False).

Operator	Name	Example	Result
==	Equal	5 == 5	True
!=	Not Equal	5 != 3	True
>	Greater Than	5 > 3	True
<	Less Than	5 < 3	False
>=	Greater or Equal	5 >= 5	True
<=	Less or Equal	5 <= 3	False

3. Logical Operators

Used to combine conditional statements (and, or, not).

Operator	Description	Example	Result
and	True if both operands are true	(5 > 3) and (2 < 4)	True
or	True if at least one operand is true	(5 > 3) or (2 > 4)	True
not	Inverts the boolean value	not (5 > 3)	False

Example:

```
a = True
b = False
print(a and b)  # False
print(a or b)   # True
```

print(not a) # False

4. Bitwise Operators

Used to perform operations on **binary numbers** (bit-level).

Operator	**Name**	**Example (**a=5**,** b=3**)**	**Binary Representation**	**Result**
&	AND	5 & 3 (0b101 & 0b011)	101 & 011 = 001	1
|	OR	5 | 3 (0b101 | 0b011)	101 | 011 = 111	7
^	XOR	5 ^ 3 (0b101 ^ 0b011)	101 ^ 011 = 110	6
~	NOT (flips bits)	~5 (~0b101)	~101 = ...11111010 (2's complement)	-6
<<	Left Shift	5 << 1 (0b101 << 1)	101 → 1010	10
>>	Right Shift	5 >> 1 (0b101 >> 1)	101 → 010	2